

WM-811K · DEEP LEARNING GROUP PROJECT

Wafer-Map Defect Classification

Engineering and Evaluating a custom CNN against an Imported baseline

[Emmanuele Pucci (939157), Takudzwa Rioga (946902), Arnold Arana Muchaypina (892579)]

1

THE PROBLEM

9-way defect classification with very imbalanced data

WM-811K is a dataset of real silicon wafer maps. Each is a grid of ternary pixels namely background, passing die, or defective die.

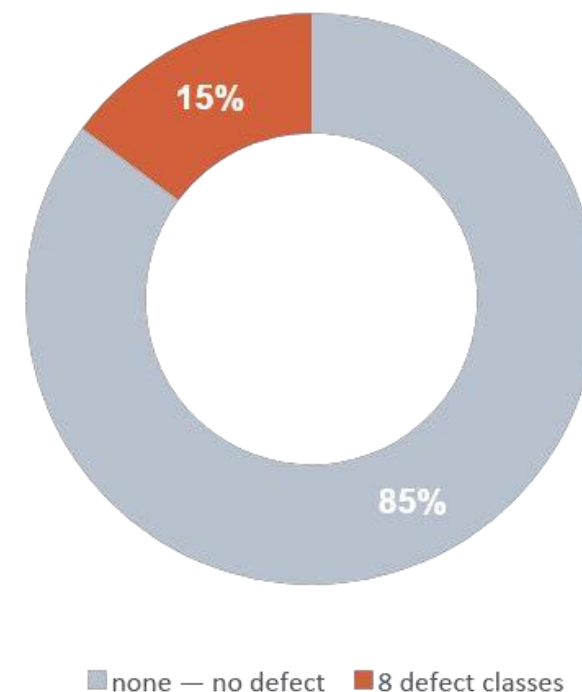
Nine pattern classes

Center · Donut · Edge-Loc · Edge-Ring · Loc · Near-full · Random · Scratch · none

The catch. “none” (no defect) is ~85% of the data. The defect classes we actually care about are scarce with Near-full has just 15 test wafers and 134 test samples.

A model can score 85% accuracy by labelling every wafer as “none”.

Test set composition (n = 17,295)



Why accuracy is the wrong score

85%

accuracy from a model that predicts “none”
every single time

The fix: macro F1

Overall accuracy is dominated by the majority class which tells you nothing about whether the defect classes we are actually trying to find have been learnt at all.

Macro F1 averages per-class F1 with equal weight, so each defect class counts the same as “none”.

We also read macro precision and recall separately. F1 alone hides in which direction a model is making errors.

Getting the representation right

1

One-hot, then resize

Interpolating raw ternary values (0/1/2) invents physically meaningless. 1.5 has no meaning in our dataset. One-hot encoding first keeps the background / pass / defect planes consistent through any resize.

2

D₄ symmetry augmentation

Dihedral-group augmentation (D₄) applies 8 rotations and reflections to our resized square wafer images. This is safe because they preserve center-vs-edge structure. An edge defect stays an edge defect and the effect is multiplying scarce minority data without distorting its label.

3

Undersample “none”

Bring the 85% majority down toward the augmented minorities, so the network trains on a balanced mix instead of having an over representation of “none” samples to learn from.

Why one-hot must come before resizing

The trap: interpolating ternary pixels

Each pixel is one of three discrete values

[0 background, 1 passing die, 2 defective die]

Resize the raw 2-D map and the interpolator averages neighbours, inventing values like **1.5** which is a pixel that is neither pass nor defect. 1.5 is physically meaningless.

Worth noting: Resizing the wafers with Nearest-Neighbours is safe. Nearest-neighbour never invents fractional pixels, which are meaningless in this context.

The fix: one-hot, then nearest-neighbour

One-hot encode first → **three binary planes** (background / pass / defect). Resize each plane with nearest-neighbour, which only ever copies existing 0s and 1s. The channels stay crisp and no fractional pixels are ever created.

background

pass die

defect

Shared across every pass. This is the one preprocessing choice we all adopted from the start. **np.eye(3)** turns the map into clean channels in a single line, before any resizing touches it.

**np = Numpy*

5

DATA · RESIZING

Standardising wildly varying map sizes

Wafer maps range from a few dozen pixels to 212×204 . The network needs one fixed input. We chose 54×54 .

~86%

padded — maps smaller than 54×54 are framed with background pixels. No information is lost.

~14%

shrunk — larger maps reduced by nearest-neighbour. Some detail is lost, so we keep this group small.

Why 54×54 and not the maximum?

Padding every map up to the largest size (212×204) would lose no information but it would balloon memory roughly $30\times$ for the half of the dataset that is small. 54×54 is the balance. Its big enough that most maps are padded (lossless), small enough to stay light in RAM. Padding uses the background channel, **[1, 0, 0]**, so the frame reads as empty wafer.

Two ways to augment the same wafers

Stochastic augmentation

- Each new copy gets a random 90° rotation + random horizontal / vertical flips.
- Defect-pixel noise where ~4–5% of defect pixels flipped as a regulariser.
- Copies per class set by an integer-division multiplier toward the majority.

Deterministic D₄ symmetry

- Every wafer has its own 8 unique transformations.
- No noise. Every transform is label-exact.
- “none” capped to (largest minority × 8) is balanced by construction.

Why the choice matters

D₄ covers every orientation exactly once with the label provably preserved. A square’s symmetries send an edge defect to a valid edge defect. The stochastic route samples orientations at random (it can duplicate or miss them), and its integer-division balancer **floors to zero augmentations** for any class already near the majority, leaving it under-sampled. The backbone is identical throughout.

7

EVOLUTION

Five passes of data preprocessing

Pass 1

First end-to-end pipeline: one-hot -> resize (54x54, nearest-neighbour) -> stochastic augmentation. Undersample to Center frequency. The original dataset with its native train/test split was used.

Pass 2

The original data and splitting pipeline from Pass 1 are applied directly to the MATLAB baseline model to establish a primary reference benchmark.

Pass 3

Dataset restructuring: the original train/test sets are combined to create a new, stratified 90:10 split. The initial stochastic preprocessing (random bit flipping) is maintained.

Pass 4

Cross-model validation: both the MATLAB baseline and the other deep learning architectures are systematically retrained and compared using the new 90:10 dataset split.

Pass 5

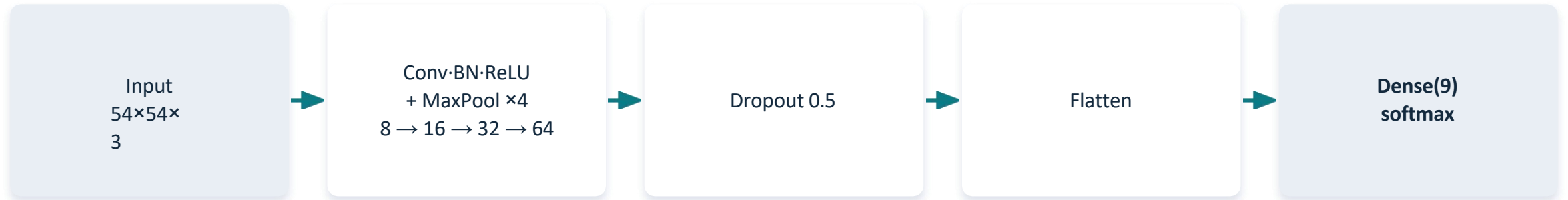
Advanced preprocessing optimization: stochastic augmentation is replaced by a deterministic pipeline based on the D4 dihedral group (rotations and reflections) to retrain the MATLAB model.

The architecture never changed. The same MATLAB-port Model runs through every pass, so each step varies only the data. That is exactly what isolates data preparation as the lever.

8

ARCHITECTURE

Ported Model Architecture



Flatten and NOT Global Average Pooling

GAP averages each feature map over height and width, discarding *where* a pattern sits. But several classes are *defined* by position. Center sits in the middle, Edge-Ring hugs the rim, Edge-Loc near the edge. Flatten preserves the spatial layout so the classifier can still read it. The small filter counts keep the model lean.

9

ARCHITECTURE

Architectures and Experimentation

With the data pipeline fixed, we tried two architectures plus three targeted enhancements grafted onto the Ported Model.

Ported Matlab Model

VGG-style · Flatten head

Conv-BN-ReLU $\times 4$ ($8 \rightarrow 16 \rightarrow 32 \rightarrow 64$), Flatten, Dense(9). Preserves spatial position. This is the basis for our enhancement experiments.

GlobalMaxPool family

Drop background $\rightarrow$ conv $\rightarrow$ GMP

Six lean variants. Small and fast, but pool position away.

Three enhancements on the Ported Model

Coordinate channels (absolute position)

Residual blocks (depth $\rightarrow$ 128)

Squeeze-and-Excitation (channel attention)

Three enhancements to the Ported Model

A

Coordinate channels

Append (x, y, radial) maps to the input so filters can read absolute position. The aim was to improve the position-defined classes.

B

Residual blocks

Skip connections + depth ($\rightarrow$ 128 filters) for a richer feature hierarchy. The intuition was depth helps the hard shape classes.

C

Squeeze-and-Excitation

Per-image channel attention and re-weight which feature maps matter for the wafer in front of it.

Controlled by construction. All three keep the Flatten head and train on the identical D₄ pipeline. Only the architecture changes, so any difference is the architecture's doing. The lean GMP family (next) is the deliberate contrast that drops position entirely.

Lean models GlobalMaxPool Models

We ran a systematic study of small GMP architectures. Four findings carried over and, on the common macro-F1 measure, the best GMP model reaches **0.830, just below the 0.890 Flatten Ported Model**.

Drop the background channel

A Lambda slice hands the conv only the pass/defect planes. The single biggest change in this experiment saves processing from being wasted modelling empty wafer.

Increased Depth

Deeper GMP variants with $\sim 2\times$ the parameters didn't beat the lean version. For this problem, bigger $\neq$ better.

GlobalMaxPool and Position Information

GlobalMaxPool keeps the model tiny but discards where a pattern sits, which is the weakness predicted for the position-defined classes.

Applying Class Weights

Adding class weights to the lean model lowered its score which is an independent confirmation of the stacking effect also seen in the Ported Model.

Our first numbers showed an Error in our Methodology

FIRST LOOK

≈ 92%

macro score obtained by the **Ported Model** on a re-balanced test split. Looked like a strong result.



THE PROBLEM

It measured per-class recall on artificially balanced data — and reported nothing about precision. Minority scores looked great because false positives were invisible.



THE FIX

Re-evaluate on the original, imbalanced test set (n = 17,295) with precision, recall and F1 per class. The picture changed and it exposed our class weights were negatively affecting our models.

Leaderboard with Macro F1 on the Imbalanced Test Set

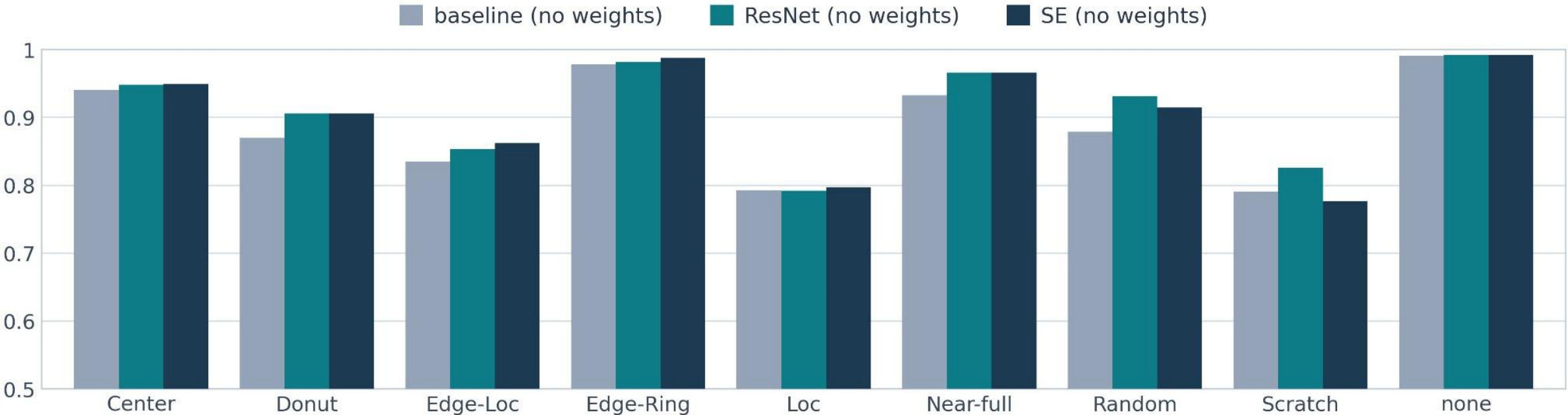
Model	Accuracy	Macro P	Macro R	Macro F1
ResNet ★	0.981	0.940	0.886	0.911
Squeeze-Excitation	0.981	0.929	0.889	0.906
CoordConv	0.975	0.875	0.914	0.890
baseline	0.978	0.896	0.886	0.890
baseline + weights	0.967	0.820	0.920	0.859
Lambda (GMP)	0.965	0.818	0.854	0.830

Read it two ways

Two custom models, ResNet and SE, performed better than the ported baseline.

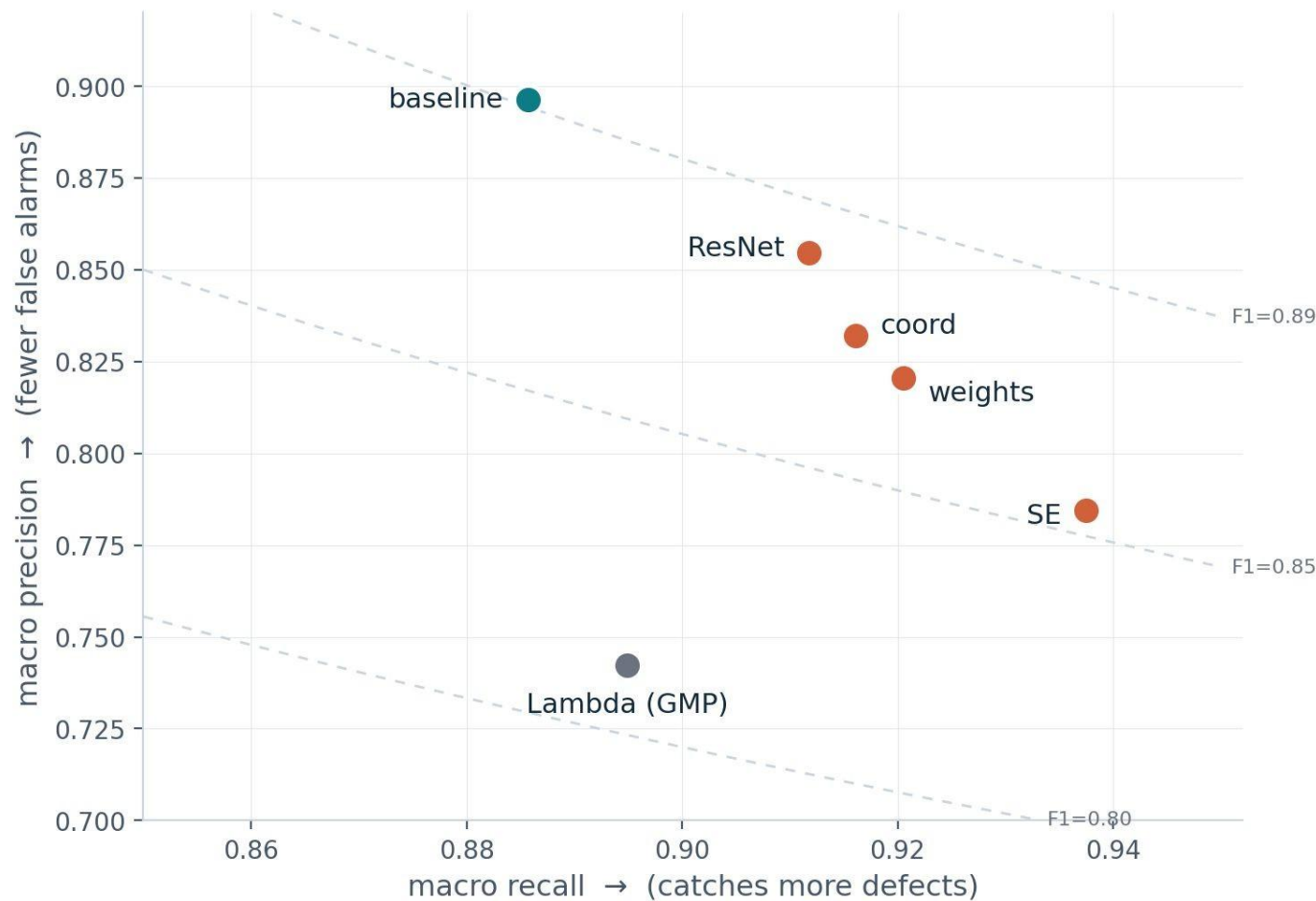
Accuracy is useless for ranking: it barely moves (0.965–0.981) while macro F1 spans 0.830–0.911.

Where the models win and lose, class by class



ResNet and SE beat or match the baseline on almost every class. Edge-Ring and none are solved everywhere (0.98–0.99). The contest is between the classes Loc, Edge-Loc, Center, Donut. ResNet is substantially better at detecting **Scratch to 0.83** (from 0.79) and leads on Random.

“Class Weights” traded precision for recall

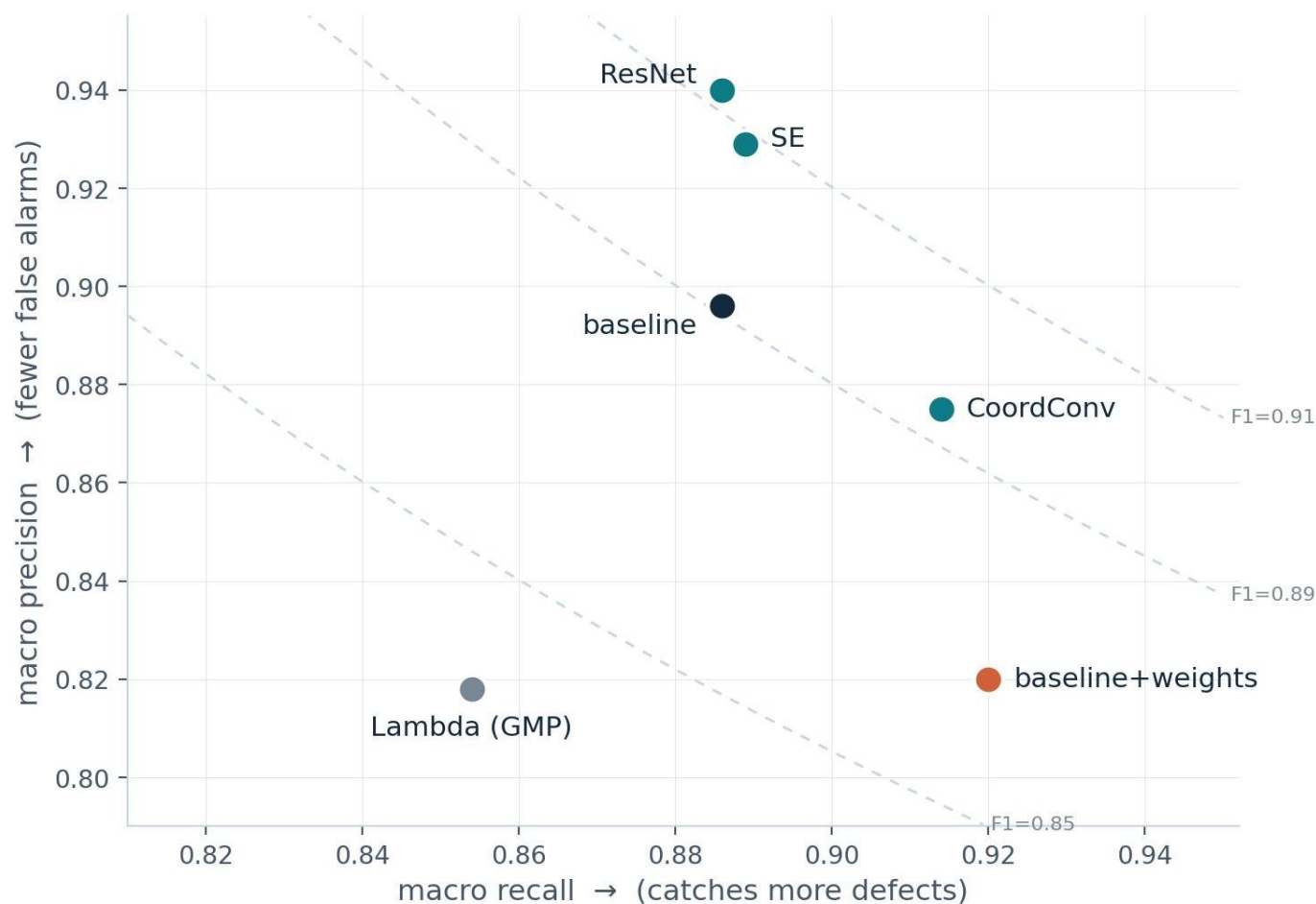


The six models sit along **one precision–recall tradeoff**.

The intervention of adjusting the initial weights pushes recall up but precision *down* and slides onto a lower F1 contour.

The plain baseline (no class weights) sits alone on the top F1 contour.

Removing Class weights lifts the architectures past the baseline



Without class weights, **ResNet and SE jump into the high-precision corner** with F1 contours above the baseline.

ResNet (0.911) and SE (0.906) now sit clear of the baseline (0.890); CoordConv lands on the same contour, trading a little precision for recall.

Class weights still drag the baseline down and to the right with high recall, poor precision, lower F1.

What the errors actually are



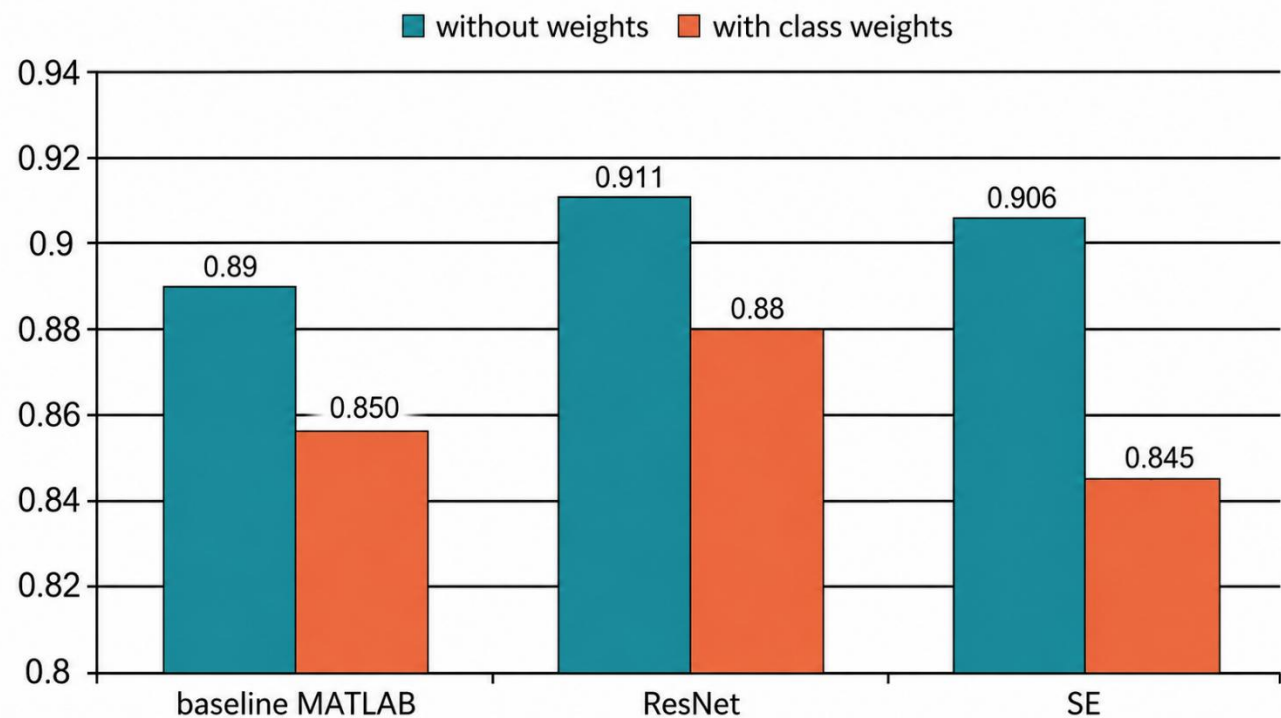
Uncertain minorities → “none”. The baseline labels 18% of Scratch and 13% of Loc as “none”. These are missed defects.

Class Weights flip the result. Scratch recall jumps to 91%, but now other classes leak into Scratch and its precision drops from 0.86 → 0.44

baseline + weights				
	precision	recall	f1-score	support
Center	0.878	0.960	0.918	429
Donut	0.857	0.857	0.857	56
Edge-Loc	0.743	0.867	0.800	519
Edge-Ring	0.976	0.979	0.978	968
Loc	0.740	0.760	0.750	359
Near-full	1.000	1.000	1.000	15
Random	0.752	0.977	0.850	87
Scratch	0.443	0.908	0.595	119
none	0.995	0.975	0.985	14743
accuracy			0.967	17295
macro avg	0.820	0.920	0.859	17295
weighted avg	0.973	0.967	0.969	17295

The stacking paradox: more techniques, worse model

The intuition: stacking proven techniques should compound into a better model. It didn't.



Lesson: *techniques compound only when they target different bottlenecks. Same-target techniques interfere.*

Why it backfired

Undersampling, augmentation and class weights all attack the **same problem (Imbalance)**. Once the data is balanced, the weights are a second correction on top of the first. They over-shoot toward the minorities and collapse precision as seen with Scratch precision falling to **0.44**.

Seen twice, independently. Our D₄ pipeline and the GMP pipeline both lost ground when weights were stacked on top of other techniques. The GMP Model dropped from 89.5% → 85.6% accuracy.

A deeper network still loses to ResNet

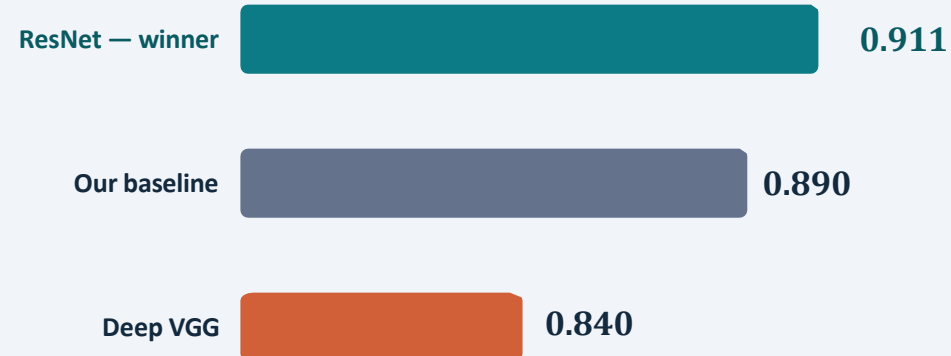
A separate set of experiments looked to build an independent pipeline around a **deeper network**. It still didn't close the gap with ResNet.

The experiment

Preprocessing: stratified 90:10 split, majority class undersampling (15k samples), minority class augmentation (salt & pepper, rotations, flips), resizing to 64×64 via nearest-neighbour interpolation, and one-hot encoding of wafer maps (3 channels)

Model: a deep 9-layer CNN (5×5 → 3×3 kernels, 32–128 filters) with BatchNorm + ReLU, two max-pooling stages, followed by flattening and a fully connected head with two 140-unit dense layers.

Macro F1 on the true test set



Depth and parameters didn't help. Without residual connections, simply stacking nine layers fails to close the gap. The deep VGG model performs the worst (0.840), trailing our baseline (0.890) and ResNet (0.911) by up to seven points in Macro F1

What actually improved performance

Class weights are a net loss

F1 Score for **baseline 0.890** vs **+weights 0.859**. This decrease in F1 was seen in *every* architecture tested. On data already balanced by undersampling + augmentation, weights are a second correction stacked on the first.

Architecture showed real gains in F1

ResNet 0.911 · SE 0.906 · CoordConv 0.890

Without weights, depth (ResNet) and channel attention (SE) clear the baseline. Coordinate channels match the baseline model. The architecture changes showed the best improvements

The decisive experiment

Architecture without weights. Every variant had been trained *with* weights, confounding it against the plain baseline. Removing class weights together with architectural changes and the D₄ data augmentation is what helped the most. **The result:** ResNet reaches **macro F1 0.911**, beating the baseline's 0.890 matching its recall while adding ~4 points of precision

Which model for production and on what metric

Don't select on

Overall accuracy

Weighted F1

- Both are dominated by “none” (85%) and cluster at 0.95–0.98
- You can't tell the models apart with accuracy alone.

Judge on

- Macro F1: primary, equal weight to every class.
- Macro precision & recall separately. F1 fixes a 50/50 cost you may not have.
- Per-class F1 for the defects that matter most operationally.
- Error cost: a missed defect vs none vs a false alarm vs a swapped defect type.

Recommendation: ResNet, with NO class weights.

It has the best macro F1 (0.911), best precision (0.940), top accuracy and it beats the baseline. Same recall as baseline with +4 points of precision, ahead on every class. SE is a close second (0.906).

What we learned

1

The metric is half the experiment. On imbalanced data, accuracy and balanced-split recall both mislead. Macro F1 on the true distribution is the honest signal.

2

Don't double-correct imbalance. Class weights on already-balanced data buy recall at a steeper precision cost and produce a net harm.

3

Position is a feature. Depth (ResNet) lifts the hard shape classes. Pooling location away (GMP) hurts the position-defined classes.

4

Data preparation is a lever. Same model, different data pipelines (stochastic vs deterministic D₄ augmentation) shift the per-class results.

What would a fab do with this information?

- Validate ResNet on a fresh wafer batch before any deployment.
- Tune the decision threshold to the fab's miss-vs-false-alarm cost.
- Try focal loss as a smoother alternative to threshold tuning.